

Source-to-equation guide to the projected-Fock scattering code

Every computational line, in execution order

Documentation for `codex/code`

Source inspected 25 September 2026

Abstract

This is a reading guide to the actual source. Each listing is copied from the current Python files; the table after it accounts for every executable statement. One table row can cover the continuation lines of a single Python statement. Comments and blank lines have no operation. Equations specify the array or matrix entries produced. The notebook section describes its current cells, including settings that differ from the function defaults.

Contents

1	The index convention used everywhere	2
1.1	Execution graph and shapes	2
2	Configuration: <code>src/xp_config.py</code>	3
3	Basis construction: <code>src/states.py:10--36</code>	3
3.1	The three dimensions	4
4	Initial ket: <code>src/states.py:39--93</code>	4
4.1	Packet and TLS coefficients	4
4.2	Number state: why <code>v[2*i:2*i+2]+=...</code>	5
4.3	Coherent state and projection	6
5	Hamiltonian assembly: <code>src/hamiltonians.py</code>	7
5.1	Mathematical target and constructor	7
5.2	Free diagonal: every assigned entry	7
5.3	Profile and every sparse transition	8
5.4	Adding the physical coupling	9
6	Grid and resource count: <code>src/experiment.py:12--57</code>	10
6.1	Signed momenta	10
6.2	Dimension formulas and memory estimate	11
7	Experiment: construction and propagation	12
7.1	Object construction, <code>src/experiment.py:60--85</code>	12
7.2	QuTiP call and every output slice, <code>src/experiment.py:87--110</code>	13
8	Scattering observables: <code>src/experiment.py:112--152</code>	14
8.1	Selecting the one-waveguide-photon sector	14
8.2	Probabilities, direction and photon-count sectors	14
8.3	Position-space one-photon amplitude	15
9	Comparing two bases: <code>src/fidelities.py</code>	16
9.1	Occupation-key map and TLS reduction	16
9.2	Preconditions, normalization and the 2×2 matrix G	17

10 Script entry point: <code>experiments/scattering.py</code>	18
11 Notebook <code>notebooks/scattering.ipynb</code> : current cells	20
11.1 Cell 1: imports and module lookup	20
11.2 Cell 2: physical inputs and advisory count	20
11.3 Cell 3: the simulation actually requested	21
11.4 Cell 4: spatial and time plots	21
11.5 Cell 6: two truncations and three fidelities	22
11.6 Cell 8: coupling sweep, currently enabled	23
12 A complete small-vector example	23
13 Imports, package markers, and files without numerical steps	24
14 What the equations certify, and what they do not	24

1 The index convention used everywhere

Symbol / Python object	Exact meaning
$M = \text{basis.modes}$	Number of retained signed field momenta. Python mode index $m = 0, \dots, M-1$.
$N = \text{basis.cap}$	Photon occupation cap; its constraint depends on <code>truncation</code> .
$k_m = \text{k_tab}[m]$	Signed wavevector; $\omega_m = k_m $ because the code uses $c = 1$.
$\mathbf{n} = (n_0, \dots, n_{M-1})$	Field occupation tuple; every tuple has exactly M entries.
i	<code>basis.index[occupation]</code> : position of a tuple in <code>basis.states</code> ; not a mode index.
$s \in \{0, 1\}$	TLS index: $s = 0$ is $ g\rangle$, $s = 1$ is $ e\rangle$.
$q = 2i + s$	Position of $ \mathbf{n}_i; s\rangle$ in a ket or Hamiltonian matrix.
$v_q = C_{i,s}$	Complex amplitude in $ \psi\rangle = \sum_{i,s} C_{i,s} \mathbf{n}_i; s\rangle$.
$B = \mathcal{B}_f $, $d = 2B$	Number of occupation tuples; ket length and Hamiltonian dimension.
L, D, ω_0, x_a	Box length, coupling parameter, TLS frequency, TLS position; keys <code>L</code> , <code>D</code> , <code>omega_0</code> , <code>x_tls</code> .
t_j	Requested output time. Index j enumerates saved states, not internal ODE steps.

The indexing identity required by almost every file is

$$\boxed{v[2i] = C_{i,g}, \quad v[2i+1] = C_{i,e}, \quad v[2i : 2i+2] = (C_{i,g}, C_{i,e}).} \quad (1)$$

Python excludes the right endpoint of a slice, so `v[2*i:2*i+2]` selects exactly two cells. Similarly, `v[0::2]` selects every even cell and `v[1::2]` every odd cell.

1.1 Execution graph and shapes

$$\begin{aligned}
&\text{four dictionaries} \xrightarrow{\text{ExperimentConfig}} \text{Experiment} \\
&\xrightarrow{\text{momentum_modes}} k \in \mathbb{R}^M \xrightarrow{\text{FockBasis}} \{\mathbf{n}_i\}_{i=0}^{B-1} \\
&\xrightarrow{\text{Hamiltonian}} H_0, V \in \mathbb{C}^{d \times d} \xrightarrow{\text{initial_state}} v_0 \in \mathbb{C}^d \\
&\xrightarrow{\text{sesolve}} v(t_j) \in \mathbb{C}^d \xrightarrow{\text{even/odd slices}} C_g, C_e \in \mathbb{C}^{J \times B} \\
&\xrightarrow{\text{compute_observables}} \mathcal{N}, P_e, T_1, R_1, P_n \in \mathbb{R}^J.
\end{aligned}$$

Here $J = \lfloor T/\Delta t \rfloor + 1$ if all states are stored. The program propagates a QuTiP ket; its $d \times d$ Hamiltonian is sparse. A global density matrix is never propagated.

2 Configuration: src/xp_config.py

```

1 from dataclasses import dataclass
2
3
4 @dataclass
5 class ExperimentConfig:
6     """Parameters of one waveguide scattering calculation."""
7
8     param_photon: dict
9     param_atom: dict
10    param_time_evol: dict
11    cutoffs: dict
12    n_max: int = 3
13    truncation: str = "full+totalcap"
14    RWA: bool = False
15    store_state: bool = True

```

Lines	Exact role
1	Import <code>dataclass</code> , which generates an initializer assigning declared fields.
4–5	Make <code>ExperimentConfig</code> such a data container.
6	Docstring only; no computation.
8	<code>param_photon</code> : k_0, σ_k, x_0 and optional field-preparation switches.
9	<code>param_atom</code> : ω_0, D, x_a, L , coupling profile, optional TLS preparation.
10	<code>param_time_evol</code> : $T, \Delta t$ and optional solver controls.
11	<code>cutoffs</code> : IR/UV bounds or explicit signed k_{modes} .
12	Default photon cap $N = 3$.
13	Default basis $\sum_m n_m \leq N$.
14	Default <code>False</code> retains counter-rotating transitions.
15	Default <code>True</code> requests the complete saved ket trajectory.

No grid, state or Hamiltonian is built in this class.

3 Basis construction: src/states.py:10--36

```

10 class FockBasis:
11     def __init__(self, modes, cap, truncation):
12         self.modes = modes
13         self.cap = cap
14         self.truncation = truncation
15         vacuum = (0,) * modes
16         if truncation == "truncated":
17             field_states = [vacuum]
18             for m in range(modes):
19                 for n in range(1, cap + 1):
20                     occupation = list(vacuum)
21                     occupation[m] = n
22                     field_states.append(tuple(occupation))
23         elif truncation == "full+totalcap":
24             field_states = []
25             for total in range(cap + 1):
26                 for occupied_modes in combinations_with_replacement(range(modes), total):
27                     field_states.append(tuple(np.bincount(occupied_modes, minlength=modes)))
28         elif truncation == "full":
29             field_states = list(product(range(cap + 1), repeat=modes))
30         else:
31             raise ValueError(f"Unknown truncation: {truncation}")
32
33         self.states = field_states
34         self.index = {occupation: i for i, occupation in enumerate(self.states)}
35         self.photon_numbers = np.array([sum(n) for n in self.states])
36         self.dim = 2 * len(self.states)

```

Lines	Exact operation / equation
10–11	<code>FockBasis(modes, cap, truncation)</code> receives M, N and the basis rule.
12–14	Store the three inputs on the object.
15	<code>(0,)*modes</code> repeats zero M times: $\mathbf{0} = (0, \dots, 0)$.
16–17	Start the one-occupied-mode basis with exactly one field vacuum.
18–19	Iterate $m = 0, \dots, M-1$ and $n = 1, \dots, N$: exactly MN pairs.
20–22	Copy vacuum, set entry m to n , append tuple $n\mathbf{e}_m$. Thus $\mathcal{B}_f^{\text{one}} = \{\mathbf{0}\} \cup \{n\mathbf{e}_m\}$ and $ \mathcal{B}_f^{\text{one}} = 1 + MN$. A tuple is hashable for the dictionary at line 34.
23–25	For total cap, iterate total photon count $r = 0, \dots, N$.
26	<code>combinations_with_replacement(range(M), r)</code> lists multisets of r mode indices; a repeated index denotes multiple photons in that mode.
27	<code>np.bincount(..., minlength=M)</code> converts the multiset into $(n_0, \dots, n_{M-1})$, $\sum_m n_m = r$; append this tuple once.
28–29	<code>product(range(N+1), repeat=M)</code> enumerates $\{0, \dots, N\}^M$: an independent cap in each mode.
30–31	Reject any unknown basis name.
33	Store the selected field tuples as <code>self.states</code> .
34	Create inverse map <code>index[n_i] = i</code> . <code>enumerate</code> supplies i ; later matrix construction uses this same map.
35	<code>photon_numbers[i] = $\sum_{m=0}^{M-1} n_m(i)$</code> .
36	Two TLS states per tuple: $d = 2 \mathcal{B}_f $.

3.1 The three dimensions

$$d_{\text{one}} = 2(1 + MN), \quad (2)$$

$$d_{\text{per}} = 2(N + 1)^M, \quad (3)$$

$$d_{\text{tot}} = 2 \sum_{r=0}^N \binom{M+r-1}{r} = 2 \binom{M+N}{N}. \quad (4)$$

For the last count, distributing r indistinguishable photons among M modes gives $\binom{M+r-1}{r}$. Alternatively add $n_{\text{slack}} = N - \sum_m n_m \geq 0$ and count nonnegative solutions of $n_0 + \dots + n_{M-1} + n_{\text{slack}} = N$. For $M = 2, N = 2$:

Rule	Field occupations	d
truncated	$(0, 0), (1, 0), (2, 0), (0, 1), (0, 2)$	10
full+totalcap	preceding five, plus $(1, 1)$	12
full	all $(n_0, n_1) \in \{0, 1, 2\}^2$	18

4 Initial ket: `src/states.py:39--93`

4.1 Packet and TLS coefficients

```

39 def initial_state(basis, k_tab, param_photon, param_atom):
40     """One incoming photon by default; optional number/coherent preparations."""
41     k0 = param_photon["k_0"]
42     sigma = param_photon["sigma_k"]
43     if k0 <= 0 or sigma <= 0:
44         raise ValueError("k_0 and sigma_k must be positive")
45
46     packet = np.exp(-(k_tab - k0) ** 2 / (4 * sigma ** 2))
47     packet = packet * np.exp(-1j * k_tab * param_photon["x_0"])
48     if param_photon.get("right_moving_only", True):
49         packet[k_tab <= 0] = 0
50     packet_norm = np.linalg.norm(packet)

```

```

51 if packet_norm == 0:
52     raise ValueError("The selected modes contain no incident packet")
53 packet /= packet_norm
54
55 atom = param_atom.get("initial_state", "g")
56 atom_coeffs = {
57     "g": (1, 0),
58     "e": (0, 1),
59     "+": (1 / np.sqrt(2), 1 / np.sqrt(2)),
60     "-": (1 / np.sqrt(2), -1 / np.sqrt(2)),
61 }
62 if atom not in atom_coeffs:
63     raise ValueError("initial_state must be g, e, +, or -")
64 atom_vector = np.asarray(atom_coeffs[atom], dtype=complex)

```

$$\tilde{c}_m = e^{-(k_m - k_0)^2 / (4\sigma_k^2)} e^{-ik_m x_0} \mathbf{1}_{k_m > 0} \quad \text{by default,} \quad c_m = \frac{\tilde{c}_m}{\sqrt{\sum_j |\tilde{c}_j|^2}}. \quad (5)$$

Lines	Exact operation / equation
39–40	Function receives basis, signed k array, photon and TLS dictionaries; docstring has no operation.
41–42	Read k_0 and σ_k .
43–44	Require $k_0 > 0, \sigma_k > 0$ for this packet convention.
46	Real Gaussian amplitude $e^{-(k_m - k_0)^2 / (4\sigma_k^2)}$ for every m . Its squared modulus has variance σ_k^2 before discrete normalization.
47	Multiply by $e^{-ik_m x_0}$; the phase moves $\sum_m c_m e^{ik_m x}$ toward x_0 . The array becomes complex.
48–49	Default <code>right_moving_only=True</code> : zero all $k_m \leq 0$. With <code>False</code> , remove the indicator in Eq. (5).
50–52	Compute $\ \tilde{c}\ _2$ and reject zero support, preventing division by zero.
53	In-place normalization gives $\sum_m c_m ^2 = 1$; subsequent <code>packet[m]</code> means c_m .
55	Read TLS preparation; absent key means <code>g</code> .
56–61	Map <code>g, e, +, -</code> to $\eta = (\eta_g, \eta_e)$: $(1, 0)$, $(0, 1)$, $(1, 1)/\sqrt{2}$, $(1, -1)/\sqrt{2}$.
62–63	Reject other TLS labels.
64	Convert η to a two-entry complex NumPy array.

4.2 Number state: why `v[2*i:2*i+2] += ...`

```

66 kind = param_photon.get("state", "number")
67 v = np.zeros(basis.dim, dtype=complex)
68 if kind == "number":
69     number = int(param_photon.get("n", 1))
70
71     if number < 1 or number > basis.cap:
72         raise ValueError("number-state n must satisfy 1 <= n <= n_max")
73
74     for m, amplitude in enumerate(packet):
75         occupation = [0] * basis.modes
76         occupation[m] = number
77         i = basis.index.get(tuple(occupation))
78         if i is not None:
79             v[2 * i : 2 * i + 2] += amplitude * atom_vector

```

Lines	Exact operation / equation
66	Absent <code>state</code> means <code>number</code> .
67	Allocate $v \in \mathbb{C}^d$, $v_q = 0$ for all q . Complex dtype retains phases.

Lines	Exact operation / equation
68–72	Select number branch; read integer n (default 1), require $1 \leq n \leq N$.
74	<code>enumerate(packet)</code> produces (m, c_m) for every retained mode.
75–76	Construct $\mathbf{n}^{(m)} = n\mathbf{e}_m$: all modes empty except m .
77	Look up $i_m = \text{basis.index.get}(\mathbf{n}^{(m)})$; missing tuple returns <code>None</code> .
78	Skip absent occupation. <code>isnotNone</code> is required because $i_m = 0$ is a valid index. Current three bases include every $n\mathbf{e}_m$ for $1 \leq n \leq N$.
79	By Eq. (1), add $c_m(\eta_g, \eta_e)$ to the two amplitudes belonging to $\mathbf{n}^{(m)}$: $C_{i_m, g} += c_m \eta_g, C_{i_m, e} += c_m \eta_e$. After the loop, $ \psi(0)\rangle = \left[\sum_m c_m n\mathbf{e}_m\rangle \right] \otimes (\eta_g g\rangle + \eta_e e\rangle)$. <code>+=</code> accumulates contributions; for $n \geq 1$ the occupations in this loop are distinct.

For default $n = 1, \eta = (1, 0)$, line 79 writes $v_{2i_m} = c_m, v_{2i_m+1} = 0$ and $|\psi(0)\rangle = \sum_m c_m |1_m; g\rangle$. For $n > 1$ the code uses $\sum_m c_m |n_m\rangle$, not $(\sum_m c_m a_m^\dagger)^n |0\rangle / \sqrt{n!}$; the latter includes cross-mode terms.

4.3 Coherent state and projection

```

80 elif kind == "coherent":
81     alpha = complex(param_photon.get("alpha", 1.0))
82     for i, occupation in enumerate(basis.states):
83         coefficient = np.exp(-abs(alpha) ** 2 / 2)
84         for m, n in enumerate(occupation):
85             coefficient *= (alpha * packet[m]) ** n / np.sqrt(factorial(n))
86             v[2 * i:2 * i + 2] = coefficient * atom_vector
87 else:
88     raise ValueError("photon state must be number or coherent")
89
90 norm = np.linalg.norm(v)
91 if norm == 0:
92     raise ValueError("The initial state has no support in this basis")
93 return qutip.Qobj(v / norm)

```

Set $\alpha_m = \alpha c_m$. The unprojected product state has coefficient

$$\langle \mathbf{n} | \boldsymbol{\alpha} \rangle = e^{-|\boldsymbol{\alpha}|^2/2} \prod_{m=0}^{M-1} \frac{(\alpha c_m)^{n_m}}{\sqrt{n_m!}}, \quad \sum_m |c_m|^2 = 1. \quad (6)$$

Lines	Exact operation / equation
80–81	Select coherent branch; read <code>alpha</code> (default 1), cast to complex.
82	Visit every retained occupation $\mathbf{n}_i$ and its index i . This loop implements projection by never visiting omitted tuples.
83	Initialize field coefficient at $e^{- \boldsymbol{\alpha} ^2/2} = \prod_m e^{- \alpha c_m ^2/2}$.
84–85	Multiply by $(\alpha c_m)^{n_m} / \sqrt{n_m!}$ for each field mode. Result is Eq. (6).
86	Assign $(C_{i, g}, C_{i, e}) = \text{coefficient}(\eta_g, \eta_e)$; each tuple occurs once.
87–88	Reject unknown preparation name.
90–92	Compute $\ v\ _2$ and reject zero support. Projection can make this norm less than one.
93	Return QuTiP ket $v / \ v\ _2 = P_{\mathcal{B}_f} \boldsymbol{\alpha}, \eta\rangle / \ P_{\mathcal{B}_f} \boldsymbol{\alpha}, \eta\rangle\ $.

Different bases can therefore produce different *normalized* coherent initial states; their fidelity may already be below one at $t = 0$.

5 Hamiltonian assembly: `src/hamiltonians.py`

5.1 Mathematical target and constructor

Write $f_m = 1$ for `flat` and $f_m = \sqrt{|k_m|}$ for `sqrt`. The finite Hamiltonian constructed by the code is

$$H(D) = H_0 + DV_1, \quad (7)$$

$$H_0 = \sum_m |k_m| a_m^\dagger a_m + \omega_0 |e\rangle\langle e|, \quad (8)$$

$$V_1 = \sum_m \left(u_m a_m + u_m^* a_m^\dagger \right) (\sigma^+ + \sigma^-), \quad u_m = \frac{if_m e^{-ik_m x_a}}{\sqrt{L}}. \quad (9)$$

Here $\sigma^+ = |e\rangle\langle g|$, $\sigma^- = |g\rangle\langle e|$. The actual operator is projected onto the enumerated finite basis: $P_{\mathcal{B}} H(D) P_{\mathcal{B}}$.

```

8 class Hamiltonian:
9     def __init__(self, basis, k_tab, param_atom, RWA=False):
10         self.basis = basis
11         self.k_tab = np.asarray(k_tab)
12         self.param_atom = param_atom
13         self.RWA = RWA
14         self.H0 = self.free()
15         self.V = self.interaction()

```

Lines	Exact operation
8–9	Constructor receives basis, signed momentum array, TLS dictionary and RWA switch.
10	Store the very same <code>FockBasis</code> whose indices define ket amplitudes.
11	Convert <code>k_tab</code> to a NumPy array so multiplication and absolute value act elementwise.
12–13	Store physical parameters and RWA flag.
14	Call <code>free()</code> immediately; store sparse diagonal H_0 .
15	Call <code>interaction()</code> immediately; store sparse unit-coupling V_1 . This construction runs before <code>sesolve</code> .

5.2 Free diagonal: every assigned entry

```

17 def free(self):
18     omega0 = self.param_atom["omega_0"]
19     energies = np.zeros(self.basis.dim)
20     for i, occupation in enumerate(self.basis.states):
21         field_energy = np.dot(np.abs(self.k_tab), occupation)
22         energies[2 * i] = field_energy
23         energies[2 * i + 1] = field_energy + omega0
24     return diags(energies, format="csr")

```

For occupation tuple $\mathbf{n}_i$ define $E_f(i) = \sum_{m=0}^{M-1} |k_m| n_m(i)$. Then

$$(H_0)_{2i,2i} = E_f(i), \quad (H_0)_{2i+1,2i+1} = E_f(i) + \omega_0. \quad (10)$$

Lines	Exact operation
17–18	Enter method and read ω_0 .
19	Allocate real array <code>energies</code> of length d , initially zero. No dense $d \times d$ matrix is allocated.

Lines	Exact operation
20	Iterate each occupation tuple $\mathbf{n}_i$ and its index i .
21	<code>np.abs(k_tab)</code> is $(k_0 , \dots, k_{M-1})$; dot product with the tuple is $E_f(i)$.
22	Position $2i$ is $ \mathbf{n}_i; g\rangle$, giving first diagonal in Eq. (10).
23	Position $2i + 1$ is $ \mathbf{n}_i; e\rangle$; add ω_0 for TLS excitation.
24	<code>diags(..., format="csr")</code> creates compressed-sparse-row diagonal H_0 .

5.3 Profile and every sparse transition

```

26 def interaction(self):
27     L = self.param_atom["L"]
28     x_tls = self.param_atom["x_tls"]
29     profile = self.param_atom["coupling"]
30     if profile == "sqrt":
31         form_factor = np.sqrt(np.abs(self.k_tab))
32     elif profile == "flat":
33         form_factor = np.ones(self.basis.modes)
34     else:
35         raise ValueError("coupling must be 'flat' or 'sqrt'")
36     g_unit = 1j * form_factor * np.exp(-1j * self.k_tab * x_tls) / np.sqrt(L)
37
38     rows, cols, values = [], [], []
39     for i, occupation in enumerate(self.basis.states):
40         for atom in (0, 1):
41             col = 2 * i + atom
42             for m in range(self.basis.modes):
43                 if occupation[m] and (not self.RWA or atom == 0):
44                     target = list(occupation)
45                     target[m] -= 1
46                     j = self.basis.index[tuple(target)]
47                     rows.append(2 * j + 1 - atom)
48                     cols.append(col)
49                     values.append(g_unit[m] * np.sqrt(occupation[m]))
50                 if not self.RWA or atom == 1:
51                     target = list(occupation)
52                     target[m] += 1
53                     j = self.basis.index.get(tuple(target))
54                     if j is not None:
55                         rows.append(2 * j + 1 - atom)
56                         cols.append(col)
57                         values.append(g_unit[m].conjugate() * np.sqrt(occupation[m] + 1))
58
59     return coo_matrix((values, (rows, cols)),
60                       shape=(self.basis.dim, self.basis.dim)).tocsr()

```

Bosonic matrix elements are

$$a_m |\mathbf{n}; s\rangle = \sqrt{n_m} |\mathbf{n} - \mathbf{e}_m; s\rangle, \quad a_m^\dagger |\mathbf{n}; s\rangle = \sqrt{n_m + 1} |\mathbf{n} + \mathbf{e}_m; s\rangle. \quad (11)$$

The TLS factor flips $s \mapsto 1 - s$. Consequently, with source column $q = 2i + s$ and target row $p = 2j + (1 - s)$,

$$V_{1,pq}^{(-)} = u_m \sqrt{n_m} \quad \text{if } \mathbf{n}_j = \mathbf{n}_i - \mathbf{e}_m, \quad (12)$$

$$V_{1,pq}^{(+)} = u_m^* \sqrt{n_m + 1} \quad \text{if } \mathbf{n}_j = \mathbf{n}_i + \mathbf{e}_m. \quad (13)$$

The matrix entry is $H_{pq} = DV_{1,pq}$; matrix convention is $H_{pq} = \langle \text{target } p | H | \text{source } q \rangle$.

Lines	Exact operation / equation
26–29	Enter interaction builder; read L, x_a and profile name.
30–31	For <code>sqrt</code> , vector $f_m = \sqrt{ k_m }$; each nonzero mode gets frequency-dependent coupling.
32–33	For <code>flat</code> , vector $f_m = 1$.

Lines	Exact operation / equation
34–35	Reject any other coupling profile.
36	Vectorized formula $u_m = i f_m e^{-i k_m x_a} / \sqrt{L}$. This is <i>unit D</i> ; line 65 inserts <i>D</i> . <code>1j</code> is Python's <i>i</i> .
38	Three empty lists will hold target rows p , source columns q , and complex entries $V_{1,pq}$.
39	Loop over every valid source occupation $\mathbf{n}_i$. This visits B tuples, not B^2 pairs.
40	Loop over source TLS index $s = 0, 1$.
41	Compute source ket index $q = 2i + s$ from Eq. (1).
42	Try a transition in each field mode $m = 0, \dots, M - 1$.
43	Downward transition requires $n_m > 0$. Without RWA allow either s . With RWA allow only $s = 0$: annihilate a photon and excite TLS, $a_m \sigma^+$.
44–45	Copy the occupation tuple and subtract one in mode m , giving $\mathbf{n} - \mathbf{e}_m$ while preserving all other fields.
46	Look up target occupation index j . Direct dictionary access is safe because all three bases are downward closed: removing one photon from any retained tuple remains retained.
47	Append target row $p = 2j + 1 - s$; $1 - s$ flips $g \leftrightarrow e$.
48	Append source column q .
49	Append $u_m \sqrt{n_m}$, exactly Eq. (12); the square root is required by $a_m n_m\rangle = \sqrt{n_m} n_m - 1\rangle$.
50	Upward transition: without RWA allow either s ; with RWA allow only $s = 1$ so TLS de-excites while a photon is created, $a_m^\dagger \sigma^-$.
51–52	Copy tuple and increment field mode m , giving $\mathbf{n} + \mathbf{e}_m$.
53	<code>index.get</code> returns <code>None</code> when the new occupation violates the chosen truncation. This realizes $P_B V P_B$ at the upper boundary.
54	Include upward edge only if target is in the basis. Again $j = 0$ is valid; testing <code>jisnotNone</code> handles it.
55–56	Append target row $2j + 1 - s$ and source column q .
57	Append $u_m^* \sqrt{n_m + 1}$, Eq. (13); complex conjugation is required for Hermiticity of $u_m a_m + u_m^* a_m^\dagger$.
59–60	Construct COO sparse matrix from triplets $(p, q, V_{1,pq})$ with shape $d \times d$, then convert to CSR for matrix-vector operations. A COO entry at <code>rows[r]</code> , <code>cols[r]</code> gets <code>values[r]</code> .

For one mode, $N = 1$, $x_a = 0$, and $f = \sqrt{k}$, the order $(|0; g\rangle, |0; e\rangle, |1; g\rangle, |1; e\rangle)$ gives

$$H = \begin{pmatrix} 0 & 0 & 0 & Du \\ 0 & \omega_0 & Du & 0 \\ 0 & Du^* & k & 0 \\ Du^* & 0 & 0 & k + \omega_0 \end{pmatrix}, \quad u = \frac{i\sqrt{k}}{\sqrt{L}}.$$

Rows and columns are displayed to expose the index rule. With RWA, only the $(|0; e\rangle, |1; g\rangle)$ block remains coupled. For a nonzero x_a , u carries the phase in Eq. (9).

5.4 Adding the physical coupling

```

62 def build_hamiltonian(self, D=None):
63     if D is None:
64         D = self.param_atom["D"]
65     return qutip.Qobj(self.H0 + D * self.V)

```

Lines	Exact operation
62	<code>D=None</code> allows either configured coupling or an explicit override.
63–64	If no override, read <code>D = param_atom["D"]</code> .
65	Form $H(D) = H_0 + DV_1$ as a sparse sum and wrap it as a QuTiP <code>Qobj</code> . No global density matrix is created.

6 Grid and resource count: `src/experiment.py:12--57`

6.1 Signed momenta

```

12 def momentum_modes(param_atom, cutoffs):
13     """Signed periodic-box momenta in the chosen IR/UV band."""
14     if "k_modes" in cutoffs:
15         k_tab = np.asarray(cutoffs["k_modes"], dtype=float)
16         if k_tab.ndim != 1 or len(k_tab) == 0 or len(np.unique(k_tab)) != len(k_tab):
17             raise ValueError("k_modes must be a nonempty list of distinct wavevectors")
18         return np.sort(k_tab)
19
20     L = param_atom["L"]
21     m_max = int(np.ceil(cutoffs["uv_cutoff"] * L / (2 * pi)))
22     k_tab = 2 * pi * np.arange(-m_max, m_max + 1) / L
23     band = (np.abs(k_tab) >= cutoffs["ir_cutoff"] - 1e-12) & \
24           (np.abs(k_tab) <= cutoffs["uv_cutoff"] + 1e-12)
25     return k_tab[band]

```

For periodic length L , the candidate momenta are $k_j = 2\pi j/L$. The automatic branch returns

$$\mathcal{K} = \left\{ \frac{2\pi j}{L} : |j| \leq j_{\max}, \quad \Omega_{\text{IR}} - 10^{-12} \leq \left| \frac{2\pi j}{L} \right| \leq \Omega_{\text{UV}} + 10^{-12} \right\}. \quad (14)$$

Lines	Exact operation
12–13	Function takes atom parameters and cutoff dictionary; docstring states signed periodic momenta.
14	If <code>cutoffs</code> contains <code>k_modes</code> , use those prescribed values instead of generating an IR/UV grid.
15	Cast explicit values to one NumPy float array. Their phases and $ k $ later use these exact numbers.
16–17	Require one-dimensional, nonempty and pairwise distinct wavevectors. <code>np.unique</code> counts exact distinct floats.
18	Sort explicit k from negative to positive; source-file mode index m follows this sorted order.
20	Read L .
21	$j_{\max} = \lceil \Omega_{\text{UV}} L / (2\pi) \rceil$. Ceiling ensures the candidate integer range reaches the UV edge.
22	<code>np.arange(-m_max, m_max+1)</code> gives all integers from $-j_{\max}$ through $+j_{\max}$, inclusive; multiply by $2\pi/L$.
23–24	Boolean mask is the intersection of lower and upper inequalities in Eq. (14). <code>&</code> is elementwise logical AND. The 10^{-12} margin protects an intended boundary from floating-point roundoff.
25	Boolean indexing <code>k_tab[band]</code> returns only retained momenta. If $\Omega_{\text{IR}} > 0$, zero is removed automatically.

In the example $L = 20\pi$, $\Omega_{\text{IR}} = 0.8$, $\Omega_{\text{UV}} = 1.2$, the spacing is $\Delta k = 0.1$ and $M = 10$:

$$\mathcal{K} = \{-1.2, -1.1, -1.0, -0.9, -0.8\} \cup \{0.8, 0.9, 1.0, 1.1, 1.2\}.$$

An explicit `k_modes` array is not checked against periodic quantization or for zero; that branch implements the supplied finite set literally.

6.2 Dimension formulas and memory estimate

```

28 def resource_estimation(param_atom, param_timeevol, cutoffs,
29                         n_max=3, truncation="full+totalcap", store_state=True):
30
31     """Count the physical basis and stored ket history before allocating them."""
32
33     M = len(momentum_modes(param_atom, cutoffs))
34     N = n_max
35     if M == 0 or N < 1:
36         raise ValueError("At least one mode and n_max >= 1 are required")
37     dimensions = {
38         "full+totalcap": 2 * comb(M + N, N),
39         "truncated": 2 * (1 + M * N),
40         "full": 2 * (N + 1) ** M,
41     }
42     if truncation not in dimensions:
43         raise ValueError(f"Unknown truncation: {truncation}")
44
45     T, dt = param_timeevol["T"], param_timeevol["dt"]
46     if T <= 0 or dt <= 0:
47         raise ValueError("T and dt must be positive")
48
49     n_times = int(np.floor(T / dt)) + 1
50     dim = dimensions[truncation]
51     history_gib = dim * 16 * (n_times if store_state else 1) / 2**30
52     feasible = dim <= 100_000 and history_gib <= 1.0
53     print(f"Modes: {M}; truncation: {truncation}; ket dimension: {dim},")
54     print(f"Output times: {n_times}; ket/history: {history_gib:.3g} GiB")
55     print(f"Feasible: {feasible} (excludes Hamiltonian and Python objects)")
56     return {"n_modes": M, "dimension": dim, "n_times": n_times,
57           "history_gib": history_gib, "feasible": feasible}

```

Lines	Exact operation / equation
28–31	Signature and docstring. <code>param_photon</code> is absent because basis size and saved-state memory do not depend on packet coefficients.
33	$M = \mathcal{K} $, computed by the same grid function used in the experiment.
34	$N = n_{\text{max}}$.
35–36	Reject $M = 0$ or $N < 1$; a photon packet needs at least one mode.
37–41	Store three exact ket dimensions derived in the basis section: $2\binom{M+N}{N}$, $2(1 + MN)$, and $2(N + 1)^M$. <code>comb</code> computes an integer binomial coefficient.
42–43	Reject a name not in the dimension dictionary.
45	Read requested final time T and output spacing Δt .
46–47	Require positive $T, \Delta t$.
49	Number of output samples $J = \lfloor T/\Delta t \rfloor + 1$, corresponding to $t_j = j\Delta t$, $j = 0, \dots, J - 1$. If $T/\Delta t$ is not integer, $t_{J-1} < T$.
50	Choose d using the basis name.
51	Complex128 amplitude is 16 bytes. Approximate ket-history storage in GiB is $16dJ/2^{30}$ if all states stored, or $16d/2^{30}$ if only final ket. This excludes sparse H , list/QuTiP overhead, NumPy copies, and observable arrays.
52	Advisory threshold: <code>feasible</code> iff $d \leq 100\,000$ and estimated history ≤ 1 GiB. This function only returns the Boolean; <code>Experiment</code> does <i>not</i> enforce it.
53–55	Print M , scheme, d , J , estimated GiB and advisory Boolean. The <code>f"..."</code> expressions format values; <code>:</code> , inserts thousands separators, <code>:.3g</code> prints three significant digits.

Lines	Exact operation / equation
56–57	Return these five numbers/Boolean in a Python dictionary for a notebook to inspect.

For the current notebook parameters $M = 10, N = 3$,

$$d_{\text{full}} = 2(4)^{10} = 2,097,152, \quad J = 281, \quad 16dJ/2^{30} \simeq 8.78 \text{ GiB.}$$

The notebook presently prints `feasible=False` but still starts that run; the call site is analyzed below.

7 Experiment: construction and propagation

7.1 Object construction, `src/experiment.py:60--85`

```

60 class Experiment:
61     """One Gaussian packet scattered by a TLS on a chosen Fock basis."""
62
63     def __init__(self, config):
64         self.param_photon = config.param_photon
65         self.param_atom = config.param_atom
66         self.param_time_evol = config.param_time_evol
67         self.n_max = config.n_max
68         self.truncation = config.truncation
69         self.RWA = config.RWA
70         self.store_state = config.store_state
71
72         self.k_tab = momentum_modes(self.param_atom, config.cutoffs)
73         self.n_modes = len(self.k_tab)
74
75         if self.n_modes == 0:
76             raise ValueError("The momentum band contains no modes")
77
78         self.basis = FockBasis(self.n_modes, self.n_max, self.truncation)
79         self.photon_numbers = self.basis.photon_numbers
80         self.hamiltonian = Hamiltonian(self.basis, self.k_tab, self.param_atom, self.RWA)
81         self.H = self.hamiltonian.build_hamiltonian()
82         self.state0 = initial_state(self.basis, self.k_tab, self.param_photon, self.param_atom)
83         self.c_g_array = None
84         self.c_e_array = None
85         self.observables = None

```

Lines	Exact operation / dependency
60–63	Class and constructor; input is one <code>ExperimentConfig</code> .
64–66	Keep direct references to the three input dictionaries; the object can read their values later.
67–70	Copy N , scheme, RWA switch, and storage switch from config.
72	Build $\mathcal{K} = (k_0, \dots, k_{M-1})$ using Eq. (14).
73	$M = \text{len}(\mathbf{k_tab})$; subsequent arrays use this <i>actual</i> mode count.
75–76	Reject a band with zero modes before trying to construct packet or Hamiltonian.
78	Construct occupation list, inverse index, photon-count array and ket dimension as described above.
79	Alias the same photon-count array for later sector masks.
80	Construct H_0 and V_1 immediately. This is where sparse matrix assembly cost occurs.
81	Form QuTiP $H = H_0 + DV_1$.

Lines	Exact operation / dependency
82	Construct and normalize initial QuTiP ket with the <i>same</i> basis ordering and k array.
83–85	Set future coefficient arrays and observables to None ; they are unavailable until propagation and measurement methods run.

No solver is run in `__init__`; however, allocating `FockBasis` and building H can already be expensive. The memory Boolean above does not stop this constructor.

7.2 QuTiP call and every output slice, `src/experiment.py:87--110`

```

87 def propagate_state(self, progress=False):
88     """Evolve the pure state; dt specifies output spacing, not solver step size."""
89     T, dt = self.param_timeevol["T"], self.param_timeevol["dt"]
90     if T <= 0 or dt <= 0:
91         raise ValueError("T and dt must be positive")
92     self.times = np.arange(int(np.floor(T / dt)) + 1) * dt
93     method = self.param_timeevol.get("method", "bdf")
94     self.result = qutip.sesolve(
95         self.H, self.state0, self.times,
96         options={"method": method,
97                 "store_states": self.store_state,
98                 "store_final_state": True,
99                 "normalize_output": False,
100                 "rtol": self.param_timeevol.get("rtol", 1e-9),
101                 "atol": self.param_timeevol.get("atol", 1e-11),
102                 "progress_bar": "tqdm" if progress else False},
103     )
104     if self.store_state:
105         vectors = np.array([state.full()[:, 0] for state in self.result.states])
106         self.c_g_array, self.c_e_array = vectors[:, 0::2], vectors[:, 1::2]
107     else:
108         vector = self.result.final_state.full()[:, 0]
109         self.c_g_array, self.c_e_array = vector[0::2], vector[1::2]
110     return self.c_g_array, self.c_e_array

```

$$i\partial_t v(t) = H v(t), \quad v(0) = v_0, \quad t_j = j\Delta t, \quad j = 0, \dots, \lfloor T/\Delta t \rfloor. \quad (15)$$

Lines	Exact operation / equation
87–88	Method and docstring; progress=False disables progress display unless requested.
89	Read $T, \Delta t$ from dictionary.
90–91	Require positive values.
92	<code>np.arange(J)*dt</code> creates $t_j = j\Delta t$. This is an <i>output</i> grid; BDF chooses internal steps independently. Last saved time is $\lfloor T/\Delta t \rfloor \Delta t$.
93	Solver method from dictionary, default "bdf" .
94–95	<code>qutip.sesolve(H, state0, times, ...)</code> numerically solves Eq. (15); input is a ket, so state dimension is d rather than d^2 .
96	Pass selected integrator method.
97	If store_state=True , store every ket $v(t_j)$; otherwise retain no trajectory list.
98	Request final ket in either storage mode.
99	normalize_output=False : do not force $\ v(t_j)\ = 1$ after numerical integration. Norm deviation remains observable.
100–101	Relative/absolute ODE tolerances, defaults $10^{-9}, 10^{-11}$. These constrain local integration error according to solver conventions, not the Fock/mode truncation error.
102	Show tqdm progress bar only if progress=True .

Lines	Exact operation / equation
103	End the solver call; <code>self.result</code> contains QuTiP states and metadata.
104–105	If full storage, convert every QuTiP ket to a one-dimensional complex NumPy vector via <code>state.full()[:,0]</code> ; stack rows to shape (J, d) . <code>[:,0]</code> removes the single-column dimension of a ket.
106	<code>vectors[:,0::2]</code> extracts $C_g(t_j, i)$ and <code>vectors[:,1::2]</code> extracts $C_e(t_j, i)$, both shape (J, B) by Eq. (1).
107–108	Final-only branch extracts one vector of shape $(d,)$.
109	Even/odd slices now yield two arrays of shape $(B,)$; no time axis.
110	Return the arrays. They are also stored as object attributes.

QuTiP’s solver errors and finite-model errors are separate:

$$\underbrace{\|v_{\text{numerical}}(t) - v_{\text{exact finite}}(t)\|}_{\text{ODE error}} \quad \text{versus} \quad \underbrace{\|v_{\text{exact finite}}(t) - \psi_{\text{target}}(t)\|}_{\text{basis/grid model error}}.$$

The saved norm diagnoses the first on the chosen finite Hamiltonian; it cannot certify that omitted Fock states or momentum modes are unimportant.

8 Scattering observables: `src/experiment.py:112--152`

8.1 Selecting the one-waveguide-photon sector

```

112 def _one_photon_indices(self):
113     """Indices of field states containing exactly one photon."""
114     return np.array([
115         i for i, _ in enumerate(self.basis.states)
116         if self.photon_numbers[i] == 1
117     ], dtype=int)

```

Lines	Exact operation
112–113	Helper function and docstring: select field occupations with one photon.
114–115	List comprehension iterates tuple index i ; the tuple itself is unused here.
116	Require $\sum_{m < M} n_m(i) = 1$. Thus there is exactly one occupied waveguide mode.
117	Return matching tuple indices as an integer NumPy array, suitable for selecting columns of C_g .

8.2 Probabilities, direction and photon-count sectors

```

119 def compute_observables(self):
120     if self.c_g_array is None:
121         raise RuntimeError("Call propagate_state first")
122     c_g = np.atleast_2d(self.c_g_array)
123     c_e = np.atleast_2d(self.c_e_array)
124     probability = np.abs(c_g) ** 2 + np.abs(c_e) ** 2
125     one_photon = self._one_photon_indices()
126     mode_index = np.array([np.argmax(self.basis.states[i][:self.n_modes])
127                             for i in one_photon], dtype=int)
128     right = one_photon[self.k_tab[mode_index] > 0]
129     left = one_photon[self.k_tab[mode_index] < 0]
130
131     self.observables = {
132         "time": self.times if self.store_state else self.times[-1:],
133         "norm": probability.sum(axis=1),
134         "p_excited": (np.abs(c_e) ** 2).sum(axis=1),
135         "p_transmitted": (np.abs(c_g[:, right]) ** 2).sum(axis=1),

```

```

136     "p_reflected": (np.abs(c_g[:, left]) ** 2).sum(axis=1),
137 }
138 for n in range(int(self.photon_numbers.max()) + 1):
139     self.observables[f"p_{n}_photon"] = probability[:, self.photon_numbers == n].sum(axis=1)
140 return self.observables

```

Let $C_{j,i,s}$ be the returned amplitudes. The exact dictionary formulas are

$$\mathcal{N}_j = \sum_i (|C_{j,i,g}|^2 + |C_{j,i,e}|^2), \quad (16)$$

$$P_{e,j} = \sum_i |C_{j,i,e}|^2, \quad (17)$$

$$T_{1,j} = \sum_{m:k_m > 0} |C_{j,i(1_m),g}|^2, \quad R_{1,j} = \sum_{m:k_m < 0} |C_{j,i(1_m),g}|^2, \quad (18)$$

$$P_{n,j} = \sum_{i: \sum_{m < M} n_m(i) = n} (|C_{j,i,g}|^2 + |C_{j,i,e}|^2). \quad (19)$$

Lines	Exact operation / equation
119–121	Reject measurement before <code>propagate_state</code> ; coefficient arrays would still be <code>None</code> .
122–123	<code>np.atleast_2d</code> turns final-only shape $(B,)$ into $(1, B)$; stored trajectory (J, B) is unchanged. Both branches can now use <code>axis=1</code> .
124	Elementwise $P_{j,i} = C_{j,i,g} ^2 + C_{j,i,e} ^2$. <code>np.abs</code> is modulus, not real part.
125	Get tuple indices with one waveguide photon.
126–127	For each such tuple, <code>np.argmax(fieldoccupations)</code> returns its unique occupied mode m because total field count is exactly one.
128	Keep one-photon tuple indices whose occupied $k_m > 0$; right-moving set.
129	Keep corresponding $k_m < 0$ indices; left-moving set. A possible $k_m = 0$ belongs to neither.
131–132	Create output dictionary; time vector is all t_j if stored or only final t_{J-1} otherwise.
133	Sum $P_{j,i}$ over <code>axis=1</code> (basis axis) to get $\mathcal{N}_j$.
134	Sum excited amplitudes over basis axis to get $P_{e,j}$.
135	Select ground-TLS amplitudes in right-moving one-photon tuples, square modulus and sum: $T_{1,j}$.
136	Same for left-moving tuples: $R_{1,j}$.
137	Close dictionary literal.
138	Loop $n = 0, \dots, \max_i \sum_{m < M} n_m(i)$. For <code>full</code> , the largest n can be MN , not N .
139	Boolean mask <code>photon_numbers==n</code> selects occupation columns with n waveguide photons. Sum their $P_{j,i}$ to get $P_{n,j}$. TLS excitation is not included in n .
140	Return and retain dictionary of time arrays.

$T_1 + R_1$ need not equal one: TLS-excited and multi-photon sectors can retain probability. These are finite-time sector probabilities; interpreting them as asymptotic scattering requires the outgoing packet to have cleared the TLS without periodic wraparound.

8.3 Position-space one-photon amplitude

```

142 def one_photon_wavefunction(self, i, x_tab):
143     """Schrodinger-picture wavefunction of the |one photon; g> sector."""
144     if self.c_g_array is None:

```

```

145         raise RuntimeError("Call propagate_state first")
146         c_g = self.c_g_array[i] if self.store_state else self.c_g_array
147         one_photon = self._one_photon_indices()
148         mode_index = np.array([np.argmax(self.basis.states[j][:self.n_modes])
149                                for j in one_photon], dtype=int)
150         x_tab = np.asarray(x_tab)
151         return np.exp(1j * np.outer(x_tab, self.k_tab[mode_index])) @ \
152             c_g[one_photon] / np.sqrt(self.param_atom["L"])

```

$$\phi_{1,g}(x, t_j) = \frac{1}{\sqrt{L}} \sum_{m=0}^{M-1} C_{j,i(1_m),g} e^{ik_m x}. \quad (20)$$

Lines	Exact operation
142–143	Method signature and picture convention in docstring.
144–145	Reject call before amplitudes exist.
146	With full storage choose row $j = i$; with final-only storage use its sole vector, so argument i has no effect in that branch.
147	Select one-waveguide-photon tuple indices.
148–149	For each tuple find its unique occupied waveguide mode m .
150	Convert requested positions x_ℓ to NumPy array.
151–152	<code>np.outer(x_tab, k_tab[mode_index])</code> makes matrix $X_{\ell m} = x_\ell k_m$; exponentiation gives $e^{iX_{\ell m}}$; <code>@</code> multiplies by coefficients $C_{j,i(1_m),g}$; divide by $\sqrt{L}$. Output is $\phi_{1,g}(x_\ell, t_j)$ for each position.

The solver already evolves Schrödinger-picture coefficients, so Eq. (20) needs no additional factor $e^{-i|k_m|t}$.

9 Comparing two bases: src/fidelities.py

For two normalized global kets A, B , three squared fidelities are returned:

$$F_{\text{state}} = |\langle \psi_A | \psi_B \rangle|^2, \quad F_{\text{atom}} = \left[\text{Tr} \sqrt{\sqrt{\rho_a^A} \rho_a^B \sqrt{\rho_a^A}} \right]^2, \quad F_{\text{photon}} = \left[\text{Tr} \sqrt{\sqrt{\rho_f^A} \rho_f^B \sqrt{\rho_f^A}} \right]^2. \quad (21)$$

The reduced matrices ρ_a and ρ_f mean partial traces over field and TLS, respectively. They are *diagnostics* derived from ket coefficients; the global evolution remains closed and pure. The code computes F_{photon} without allocating a large field density matrix.

9.1 Occupation-key map and TLS reduction

```

7 def _components(experiment, vector):
8     amplitudes = {}
9     for i, occupation in enumerate(experiment.basis.states):
10         amplitudes[occupation] = vector[2 * i : 2 * i + 2]
11     return amplitudes
12
13
14 def _atom_density_matrix(vector):
15     coefficients = vector.reshape(-1, 2)
16     return coefficients.T @ coefficients.conj()

```

Lines	Exact operation / equation
7–8	Define the helper and make an empty dictionary: occupation key $\mapsto$ the two amplitudes (C_g, C_e) .

Lines	Exact operation / equation
9	Visit each occupation $\mathbf{n}_i$ in this experiment's own basis ordering.
10	Save two-component vector <code>vector[2*i:2*i+2]</code> under the field occupation tuple, using Eq. (1). This changes labels without changing amplitudes.
11	Return occupation-to-amplitude dictionary.
14–15	Define the TLS reduction helper and reshape a length- d ket into $B \times 2$ coefficient matrix C_{is} ; <code>-1</code> asks NumPy to infer $B = d/2$.
16	<code>coefficients.T@coefficients.conj()</code> yields $(\rho_a)_{ss'} = \sum_i C_{is} C_{is'}^*$. It is 2×2 , Hermitian, trace one for a normalized ket. <code>.T</code> transposes; <code>.conj()</code> conjugates entrywise.

The ordering of raw vectors differs among truncations. Comparing entry number q across two vectors would generally compare different physical occupations. The dictionary at line 10 makes equality of keys, rather than equality of integer positions, the criterion for a physical overlap.

9.2 Preconditions, normalization and the 2×2 matrix G

```

19 def fidelities_over_time(experiment_a, experiment_b):
20     """Return exact squared fidelities on a common momentum grid."""
21     if not experiment_a.store_state or not experiment_b.store_state:
22         raise ValueError("Both experiments must store states")
23     if not np.array_equal(experiment_a.k_tab, experiment_b.k_tab):
24         raise ValueError("The two experiments must use the same signed k grid")
25     if not np.array_equal(experiment_a.times, experiment_b.times):
26         raise ValueError("The two experiments must use the same output times")
27
28     state, atom, photon = [], [], []
29     for ket_a, ket_b in zip(experiment_a.result.states, experiment_b.result.states):
30         v_a, v_b = ket_a.full()[:, 0], ket_b.full()[:, 0]
31         v_a = v_a / np.linalg.norm(v_a)
32         v_b = v_b / np.linalg.norm(v_b)
33         a = _components(experiment_a, v_a)
34         b = _components(experiment_b, v_b)
35         G = np.zeros((2, 2), dtype=complex)
36         for key in a.keys() & b.keys():
37             G += np.outer(a[key].conj(), b[key])
38         state.append(abs(np.trace(G)) ** 2)
39         photon.append(np.linalg.svd(G, compute_uv=False).sum() ** 2)
40         rho_a = qutip.Qobj(_atom_density_matrix(v_a))
41         rho_b = qutip.Qobj(_atom_density_matrix(v_b))
42         atom.append(qutip.fidelity(rho_a, rho_b) ** 2)
43     return {"state": np.array(state), "atom": np.array(atom),
44           "photon": np.array(photon)}

```

In a common field space, decompose

$$|\psi_A\rangle = |\phi_g^A\rangle|g\rangle + |\phi_e^A\rangle|e\rangle, \quad |\psi_B\rangle = |\phi_g^B\rangle|g\rangle + |\phi_e^B\rangle|e\rangle. \quad (22)$$

Define the 2×2 cross-overlap matrix

$$G_{ss'} = \langle \phi_s^A | \phi_{s'}^B \rangle = \sum_{\mathbf{n} \text{ common}} C_{A,\mathbf{n},s}^* C_{B,\mathbf{n},s'}, \quad s, s' \in \{g, e\}. \quad (23)$$

An occupation absent from one basis has zero amplitude after embedding, hence only keys in the intersection contribute.

Lines	Exact operation / equation
19–20	Function compares two completed experiments; docstring specifies squared fidelities.

Lines	Exact operation / equation
21–22	Require stored trajectories, since the function returns values at every t_j . Final-only objects do not have those lists.
23–24	Require exactly equal signed arrays k_m in the same order. A tuple entry n_m must denote the same physical mode in both experiments.
25–26	Require exactly equal output-time arrays; <code>zip</code> below then pairs states at the same time.
28	Initialize three Python lists to collect scalar fidelity values over time.
29	<code>zip</code> pairs A and B QuTiP kets at corresponding output indices.
30	<code>ket.full()</code> is a two-dimensional column array; <code>[:,0]</code> yields a one-dimensional vector.
31–32	Divide each vector by its Euclidean norm. Fidelity is defined for normalized states; this removes small ODE norm drift <i>for metrics only</i> . Observable <code>norm</code> remains raw.
33–34	Convert each vector into occupation-key dictionary with aligned TLS pairs.
35	Allocate complex 2×2 matrix $G = 0$.
36	Python <code>a.keys() & b.keys()</code> is set intersection: iterate occupations physically present in both bases.
37	<code>np.outer(a[key].conj(), b[key])</code> has entry $C_{A,n,s}^* C_{B,n,s'}$. Sum over keys yields Eq. (23).
38	$\text{Tr } G = \sum_s \langle \phi_s^A \phi_s^B \rangle = \langle \psi_A \psi_B \rangle$, so append $ \text{Tr } G ^2 = F_{\text{state}}$.
39	<code>np.linalg.svd(G, compute_uv=False)</code> gives the two singular values σ_1, σ_2 . Append $(\sigma_1 + \sigma_2)^2 = F_{\text{photon}}$, the exact Uhlmann fidelity between the reduced field states.
40–41	Construct each 2×2 TLS reduced matrix using line 16, wrapped for QuTiP.
42	QuTiP returns root fidelity $\text{Tr } \sqrt{\sqrt{\rho_a^A} \rho_a^B \sqrt{\rho_a^A}}$; square it to match Eq. (21).
43–44	Convert lists to NumPy arrays of length J and return dictionary with keys <code>state</code> , <code>atom</code> , <code>photon</code> .

Why line 39 is exact: $\rho_f^A = |\phi_g^A\rangle\langle\phi_g^A| + |\phi_e^A\rangle\langle\phi_e^A|$ has rank at most two, and likewise for B . The nonzero singular values needed for Uhlmann fidelity are those of the 2×2 cross Gram matrix G . Thus no $B \times B$ density matrix or d^2 global density matrix is needed. The ratio $F_{\text{state}}/F_{\text{atom}}$ used elsewhere is not this fidelity.

10 Script entry point: experiments/scattering.py

```

1 """Small script entry point; the same function is usable from a notebook."""
2
3 import csv
4 import sys
5 from pathlib import Path
6
7 import numpy as np
8
9 project_root = Path(__file__).resolve().parents[1]
10 sys.path.insert(0, str(project_root))
11
12 from src.xp_config import ExperimentConfig
13 from src.experiment import Experiment
14
15
16 def run_scattering(param_photon, param_atom, param_timeevol, cutoffs,
17                   n_max=3,
18                   truncation="full+totalcap",
19                   RWA=False,
20                   store_state=True,
21                   store_results=False,
```

```

22         progress=False):
23
24     """Propagate one chosen truncation and return the Experiment object."""
25
26     config = ExperimentConfig(param_photon=param_photon,
27                               param_atom=param_atom,
28                               param_timeevol=param_timeevol,
29                               cutoffs=cutoffs,
30                               n_max=n_max,
31                               truncation=truncation,
32                               RWA=RWA,
33                               store_state=store_state)
34
35     experiment = Experiment(config)
36     experiment.propagate_state(progress=progress)
37     observables = experiment.compute_observables()
38
39     if store_results:
40         out = project_root / "results" / "csv_files" / "scattering.csv"
41         out.parent.mkdir(parents=True, exist_ok=True)
42         with out.open("w", newline="") as handle:
43             writer = csv.writer(handle)
44             writer.writerow(observables)
45             writer.writerows(zip(*observables.values()))
46     return experiment

```

Lines	Exact operation
1–7	Docstring and imports: <code>csv</code> writes optional output; <code>sys</code> and <code>Path</code> locate modules/files; <code>np</code> is only used by the currently commented example.
9	<code>Path(__file__).resolve().parents[1]</code> is the <code>codex/code</code> project root.
10	Insert project root at the front of Python's import path.
12–13	Import configuration and experiment classes after path insertion.
16–22	Four required dictionaries; optional N , truncation, RWA, state storage, CSV writing and progress. Defaults: $N = 3$, <code>full+totalcap</code> , <code>RWA=False</code> , <code>store_state=True</code> , no file, no progress bar.
24	Docstring: return an <code>Experiment</code> object.
26–33	Construct <code>ExperimentConfig</code> with explicit keyword assignments. <code>store_results</code> and <code>progress</code> control I/O, not physics, and are not in the config.
35	Constructor builds grid, basis, H_0 , V_1 , H , and initial ket.
36	QuTiP propagates Eq. (15); save amplitudes.
37	Compute Eq. (19); dictionary also stored on the experiment.
39	Enter optional CSV branch only if <code>store_results=True</code> .
40	Set output path <code>results/csv_files/scattering.csv</code> under project root.
41	Create parent directories as needed; existing directories are accepted.
42	Open file for writing; existing CSV at this path is replaced. <code>newline=""</code> delegates line endings to CSV module.
43	Create a CSV writer.
44	Write metric names as header in dictionary insertion order.
45	<code>zip(*observables.values())</code> transposes metric arrays: row j contains all values at t_j .
46	Return complete experiment for notebook inspection.

Lines 49–62 are presently *comments*. Running `python3experiments/scattering.py` currently defines the function and exits without a simulation. The README's run command describes an earlier version; the active calls are in the notebook.

11 Notebook notebooks/scattering.ipynb: current cells

Cell numbers below start at zero, matching their JSON order. Markdown cells 0, 5 and 7 are text only. Stored notebook *outputs* are historical; a source cell may have changed since they were generated. Outputs of comparison cells 6 and 8 were cleared when the basis options changed.

11.1 Cell 1: imports and module lookup

```

1 import sys
2 from pathlib import Path
3 import numpy as np
4 import matplotlib.pyplot as plt
5 project_root = Path.cwd().parent
6 sys.path.insert(0, str(project_root))
7 from src.experiment import resource_estimation
8 from src.fidelities import fidelities_over_time
9 from experiments.scattering import run_scattering

```

Statement	Exact role
<code>import sys</code>	Access module search path.
<code>Path</code>	<code>from pathlib import Path</code> : filesystem path object.
<code>import numpy as np</code>	Short name for arrays, π , means, nearest-index selection and geometric grids.
<code>import matplotlib.pyplot as plt</code>	Short name for plotting.
<code>Path.cwd().parent</code>	Assumes the working directory is <code>codex/code/notebooks</code> ; parent is the project root.
<code>sys.path.insert</code>	Put that root first in Python's import search list.
<code>resource_estimation</code>	Import M, d, J and memory estimator.
<code>fidelities_over_time</code>	Import exact comparison of two trajectories.
<code>run_scattering</code>	Import construction, propagation and measurement wrapper.

11.2 Cell 2: physical inputs and advisory count

```

1 param_photon = {'k_0': 1.0, 'sigma_k': 0.10, 'x_0': -12.0}
2 param_atom = {'omega_0': 1.0, 'D': 0.45, 'x_tls': 0.0,
3              'L': 20*np.pi, 'coupling': 'sqrt'}
4 param_time_evol = {'T': 28.0, 'dt': 0.1}
5 cutoffs = {'ir_cutoff': 0.8, 'uv_cutoff': 1.2}
6 n_max = 3
7 truncation = 'full'
8 estimate = resource_estimation(param_atom, param_time_evol, cutoffs,
9                               n_max=n_max, truncation=truncation)

```

Statement	Exact meaning / consequence
<code>param_photon</code>	Eq. (5): $k_0 = 1$, $\sigma_k = 0.1$, $x_0 = -12$. Absent <code>state</code> means one photon; absent <code>right_moving_only</code> means $k > 0$ only.
<code>param_atom</code>	$\omega_0 = 1$, $D = 0.45$, $x_a = 0$, $L = 20\pi$, $f_m = \sqrt{ k_m }$. Absent <code>initial_state</code> means ground TLS.
<code>param_time_evol</code>	$T = 28$, $\Delta t = 0.1$; $J = 281$ saved times $0, 0.1, \dots, 28$.
<code>cutoffs</code>	Eq. (14): $M = 10$ signed modes, five per direction.
<code>n_max</code>	$N = 3$.
<code>truncation</code>	Current value 'full' independently caps ten modes at 3: $d = 2(4)^{10} = 2,097,152$. Function default <code>full+totalcap</code> would give $2\binom{13}{3} = 572$.

Statement	Exact meaning / consequence
<code>estimate</code>	Prints approximately 8.78 GiB for stored complex amplitudes and <code>feasible=False</code> . It does not prevent execution.

11.3 Cell 3: the simulation actually requested

```

1 experiment = run_scattering(param_photon, param_atom,
2                             param_time_evol, cutoffs,
3                             n_max=n_max, truncation=truncation,
4                             RWA=True, progress=True)
5 for name, values in experiment.observables.items():
6     print(name, values[-1])

```

Statement	Exact meaning / consequence
<code>run_scattering(...)</code>	Construct and solve with cell-2 inputs. This cell does <i>not</i> test <code>estimate['feasible']</code> ; it can begin a very large allocation.
<code>RWA=True</code>	Override default <code>False</code> ; this cell uses only energy-exchange transitions. It is a rotating-wave calculation despite the notebook title.
<code>progress=True</code>	Enable progress bar, with no change in physics.
<code>for name, values</code>	Iterate output dictionary, one metric array at a time.
<code>values[-1]</code>	Python index -1 means the last <i>time</i> sample, here $t = 28$.

11.4 Cell 4: spatial and time plots

```

1 if experiment is not None:
2     time = experiment.observables['time']
3     x_tab = np.linspace(-param_atom['L']/2, param_atom['L']/2,
4                         500, endpoint=False)
5     t_hit = param_atom['x_tls'] - param_photon['x_0']
6     snapshots = [0, np.argmin(np.abs(time - t_hit)), len(time)-1]
7     fig, axes = plt.subplots(1, 2, figsize=(11, 4))
8     for j in snapshots:
9         phi_x = experiment.one_photon_wavefunction(j, x_tab)
10        axes[0].plot(x_tab, np.abs(phi_x)**2, label=f't = {time[j]:.1f}')
11    axes[0].axvline(param_atom['x_tls'], color='black', ls=':', label='TLS')
12    axes[0].set(xlabel='x', ylabel=r'$|\phi_{1,g}(x,t)|^2$')
13    axes[0].legend()
14    for name in ('p_transmitted', 'p_reflected', 'p_excited', 'norm'):
15        axes[1].plot(time, experiment.observables[name], label=name)
16    axes[1].set(xlabel='t', ylabel='population', ylim=(0, 1.05))
17    axes[1].legend()
18    fig.tight_layout()
19    plt.show()

```

Statement	Mathematical meaning
<code>if experiment is not None</code>	Plot if a previous cell assigned an experiment; a failed previous cell can leave a stale notebook variable.
<code>time</code>	Read solver output array (t_j).
<code>x_tab</code>	Sample 500 points on $[-L/2, L/2]$; <code>endpoint=False</code> avoids duplicating periodic endpoints.
<code>t_hit</code>	$x_a - x_0 = 12$, rough flight time at speed $c = 1$, not a computed interaction time.
<code>snapshots</code>	Indices 0, nearest saved time to 12 via <code>argmin(abs(...))</code> , and $J - 1$.
<code>plt.subplots</code>	Allocate one figure and two axes.

Statement	Mathematical meaning
forj, phi_x	Compute Eq. (20) at three chosen t_j .
axes[0].plot	Plot $ \phi_{1,g}(x, t_j) ^2$ and formatted time label.
axvline, first set, legend	Mark TLS position x_a ; label spatial axes/curves. No new physics is calculated.
forname, axes[1].plot	Plot $T_1(t_j), R_1(t_j), P_e(t_j), \mathcal{N}(t_j)$ from Eq. (19).
second set, legend	Label time plot and restrict visible vertical range to (0, 1.05).
tight_layout, show	Arrange and display; no raw data file saved.

11.5 Cell 6: two truncations and three fidelities

```

1 schemes = ('full+totalcap', 'truncated')
2 runs = {}
3 for scheme in schemes:
4     estimate = resource_estimation(param_atom, param_timeevol,
5                                   cutoffs, n_max=n_max, truncation=scheme)
6     if estimate['feasible']:
7         runs[scheme] = run_scattering(param_photon, param_atom,
8                                       param_timeevol, cutoffs,
9                                       n_max=n_max, truncation=scheme)
10 reference = runs.get('full+totalcap')
11 if reference is not None:
12     fig, axes = plt.subplots(1, 3, figsize=(13, 3.5))
13     for scheme in ('truncated',):
14         if scheme in runs:
15             F = fidelities_over_time(runs[scheme], reference)
16             for ax, name in zip(axes, ('state', 'atom', 'photon')):
17                 ax.plot(reference.times, F[name], label=scheme)
18                 ax.set(xlabel='t', ylabel=f'{name} fidelity',
19                       ylim=(0, 1.05))
20                 ax.set_title(name)
21                 print(scheme, 'initial:', F['state'][0],
22                       'time mean:', {key: np.mean(value)
23                                     for key, value in F.items()})
24 axes[0].legend()
25 fig.tight_layout()
26 plt.show()

```

Statement	Exact operation
schemes, runs	List two finite bases; initialize name-to-experiment dictionary.
forscheme, estimate	Recalculate d, J for each basis with the same physical dictionaries.
ifestimate['feasible']	This cell <i>does</i> obey the advisory threshold.
runs[scheme]	Solve feasible cases. No RWA keyword is supplied, so wrapper default False applies: these runs include counter-rotating terms, unlike cell 3.
reference=...	Retrieve total-cap result, or None if skipped.
ifreference..., subplots	Make three axes only if reference exists.
forscheme, ifschemeinruns	Consider each available candidate against that same finite reference.
F=...	Compute all $F_\mu(t_j)$ in Eq. (21); requires common k array and t array.
zip(axes,...)	Pair axis 0/1/2 with names state/atom/photon .
ax.plot, ax.set, ax.set_title	Plot $F_\mu(t_j)$, label axes and use visible range (0, 1.05).
F['state'][0]	Initial global fidelity $F_{\text{state}}(0)$.
np.mean(value)	Arithmetic sampled-time mean $J^{-1} \sum_{j=0}^{J-1} F_\mu(t_j)$ for each metric.

Statement	Exact operation
<code>legend,</code> <code>tight_layout,</code> <code>show</code>	Display comparison; no file saved.

11.6 Cell 8: coupling sweep, currently enabled

```

1 RUN_SWEEP = True
2 if RUN_SWEEP:
3     D_values = np.geomspace(0.02, 0.45, 6)
4     sweep = {scheme: [] for scheme in ('truncated',)}
5     for D in D_values:
6         atom_here = dict(param_atom, D=float(D))
7         ref = run_scattering(param_photon, atom_here, param_timeevol,
8                             cutoffs, n_max=n_max, truncation='full+totalcap')
9         for scheme in sweep:
10             candidate = run_scattering(param_photon, atom_here,
11                                       param_timeevol, cutoffs,
12                                       n_max=n_max, truncation=scheme)
13             F = fidelities_over_time(candidate, ref)
14             sweep[scheme].append(np.mean(F['state']))
15     for scheme, values in sweep.items():
16         plt.semilogx(D_values, values, 'o-', label=scheme)
17     plt.xlabel('D')
18     plt.ylabel('time-mean full-state fidelity')
19     plt.legend()
20     plt.show()

```

Statement	Exact operation
<code>RUN_SWEEP=True, if</code>	Current value enters the sweep branch. Notebook Markdown says to enable it, but it is already enabled.
<code>D_values</code>	Six geometrically spaced D : equal spacing in $\log D$ between 0.02 and 0.45.
<code>sweep</code>	One empty output list keyed by candidate basis.
<code>forD, atom_here</code>	For each D , shallow-copy atom dictionary and replace only D ; original <code>param_atom</code> remains unchanged.
<code>ref</code>	One total-cap simulation at this D and default <code>RWA=False</code> .
<code>forscheme, candidate</code>	One candidate simulation per D : $6(1 + 1) = 12$ solver calls total, each rebuilding a matrix.
<code>F, append</code>	Compute full-state fidelity trajectory and append its time mean.
<code>forscheme, values,</code> <code>semilogx</code>	Plot six means against logarithmic D axis.
<code>xlabel, ylabel, legend,</code> <code>show</code>	Label/display; no raw sweep array saved.

12 A complete small-vector example

Take $M = 2, N = 1$, `truncated`, normalized packet $c = (c_0, c_1)$, and TLS spinor $\eta = (\eta_g, \eta_e)$. The basis constructor stores occupation tuples in order

$$\begin{aligned} \text{basis.states} &= [(0,0), (1,0), (0,1)], \\ \text{basis.index}[(0,0)] &= 0, \quad \text{basis.index}[(1,0)] = 1, \quad \text{basis.index}[(0,1)] = 2. \end{aligned}$$

Consequently the full ket vector has six cells:

q	0	1	2	3	4	5
ket	$ 0, 0; g\rangle$	$ 0, 0; e\rangle$	$ 1, 0; g\rangle$	$ 1, 0; e\rangle$	$ 0, 1; g\rangle$	$ 0, 1; e\rangle$
v_q	0	0	$c_0\eta_g$	$c_0\eta_e$	$c_1\eta_g$	$c_1\eta_e$

The `form, amplitude enumerate(packet)` loop writes

$$\begin{aligned} m = 0 : \quad i = 1, \quad v[2 : 4] &+= c_0(\eta_g, \eta_e), \\ m = 1 : \quad i = 2, \quad v[4 : 6] &+= c_1(\eta_g, \eta_e). \end{aligned}$$

The vacuum cells remain zero because a one-photon number state has no vacuum component. Its norm is

$$\|v\|^2 = |c_0|^2(|\eta_g|^2 + |\eta_e|^2) + |c_1|^2(|\eta_g|^2 + |\eta_e|^2) = 1.$$

This is the literal contents of the array after line 79 of `src/states.py`; the final normalization at line 93 leaves it unchanged up to floating-point roundoff.

13 Imports, package markers, and files without numerical steps

The executable listings above start at the classes/functions. The leading imports select the following exact implementations:

Source lines	Purpose
<code>src/states.py:1--7</code>	Module docstring; <code>itertools.combinations_with_replacement</code> and <code>product</code> enumerate occupations; <code>math.factorial</code> supplies $n!$ in Eq. (6); NumPy makes arrays; QuTiP wraps final ket.
<code>src/hamiltonians.py:1--5</code>	Module docstring; NumPy makes f_m and phases; QuTiP wraps H ; SciPy <code>coo_matrix</code> stores triplets and <code>diags</code> stores H_0 .
<code>src/experiment.py:1--9</code>	Module docstring; <code>comb</code> gives binomial count, <code>pi</code> the momentum spacing, NumPy arrays, QuTiP solver, and imports the local basis/Hamiltonian functions.
<code>src/fidelities.py:1--4</code>	Module docstring; NumPy computes overlaps/SVD; QuTiP computes TLS Uhlmann fidelity.
<code>src/__init__.py</code> , <code>experiments/__init__.py</code> <code>requirements.txt</code>	Empty package-marker files. No numerical statements.
<code>README.md</code>	Declares NumPy, SciPy, QuTiP, Matplotlib dependency ranges; it supplies no physical parameter.
<code>results/</code>	Short usage overview. Its script command is currently stale because the script's example is commented out.
	Optional CSV output location; no result is written unless <code>store_results=True</code> .

14 What the equations certify, and what they do not

Exact finite-code identity	Scope
$H = H_0 + DV_1$, Eqs. (7)–(9)	Sparse projected Hamiltonian for the chosen k and Fock basis.
$H^\dagger = H$ for real $D, L > 0, \omega_0$	Down/up entries are conjugate; exact finite evolution is unitary. A high norm cannot establish basis convergence.
Default $v_0 = \sum_m c_m 1_m; g\rangle$	A right-moving, one-photon preparation. Coherent preparation is normalized <i>after</i> projection.
Eq. (19)	T_1, R_1 are ground-TLS, one-photon sector weights at finite time; they are not automatically asymptotic scattering coefficients.
$F_{\text{photon}} = (\sigma_1(G) + \sigma_2(G))^2$	Exact field reduced-state fidelity on matching k, t grids, without a field density matrix.

Exact finite-code identity	Scope
<code>store_state=False</code>	Only final coefficients/observables are retained; time-dependent fidelity function requires full stored trajectories.

Numerical convergence has independent axes: photon cap N , mode band, periodic length L , and ODE tolerances. Changing one while fixing the others tests that axis only. The current notebook cell 3 uses RWA and an enormous `full` basis; cells 6 and 8 use no-RWA by default. Their curves do not represent one single parameter regime unless those switches are aligned.